

Trabajo práctico N°8



Integrantes:

- Medina, Gabriel 14086
- Vasquez, Nicolas 14097
- Albanez, Luciano 13185
- Fattiboni, Álvaro 13201

Ejercicio 1

Un fabricante vende cajas con 1000 tornillos; existen cinco tipos de cajas (hipótesis) con diferentes proporciones de recubrimiento. Las proporciones y la frecuencia con la que se llenan las cajas son:

H_a : Todos níquel. — Prior $P(H_a) = 0.15$. $P(\text{cobre} | H_a) = 0.00$

H_b : 70% níquel, 20% cobre, 10% cincado. — Prior $P(H_b) = 0.15$.
 $P(\text{cobre} | H_b) = 0.20$

H_c : 50% níquel, 25% cobre, 25% cincado. — Prior $P(H_c) = 0.50$.
 $P(\text{cobre} | H_c) = 0.25$

H_d : 20% níquel, 50% cobre, 30% cincado. — Prior $P(H_d) = 0.10$.
 $P(\text{cobre} | H_d) = 0.50$

H_e : Todos cobre. — Prior $P(H_e) = 0.10$. $P(\text{cobre} | H_e) = 1.00$

1.1 Distribución a priori sobre las hipótesis

$P(H_a) = 0.15$, $P(H_b) = 0.15$, $P(H_c) = 0.50$, $P(H_d) = 0.10$, $P(H_e) = 0.10$.

1.2 Posterior dado que los primeros 10 tornillos son cobre

Inteligencia Artificial 2025 - Grupo 5

Se asume independencia y que la probabilidad de cobre bajo cada hipótesis es la fracción indicada. La verosimilitud es p_H^{10} y se aplica Bayes: $P(H|D) \propto P(H) p_H^{10}$, normalizando luego.

Tabla de resultados:

Hipótesis	Prior	$p(\text{cobre} H)$	Verosimilitud (p_H^{10})	Prior * Verosimilitud	Posterior (dado 10 cobre)
Ha	0.150000	0.000000	0.000000e+00	0.000000e+00	0.000000
Hb	0.150000	0.200000	1.024000e-07	1.536000e-08	0.000000
Hc	0.500000	0.250000	9.536743e-07	4.768372e-07	0.000005
Hd	0.100000	0.500000	9.765625e-04	9.765625e-05	0.000976
He	0.100000	1.000000	1.000000e+00	1.000000e-01	0.999019

Interpretación: después de observar 10 tornillos todos de cobre, la hipótesis más plausible con abrumadora probabilidad es H_e (todos cobre). Las demás hipótesis quedan con probabilidades prácticamente nulas.

1.3 Gráficos

Figura 1: Evolución de la verosimilitud $P(\text{datos}|H)$ cuando los primeros n tornillos son todos cobre.

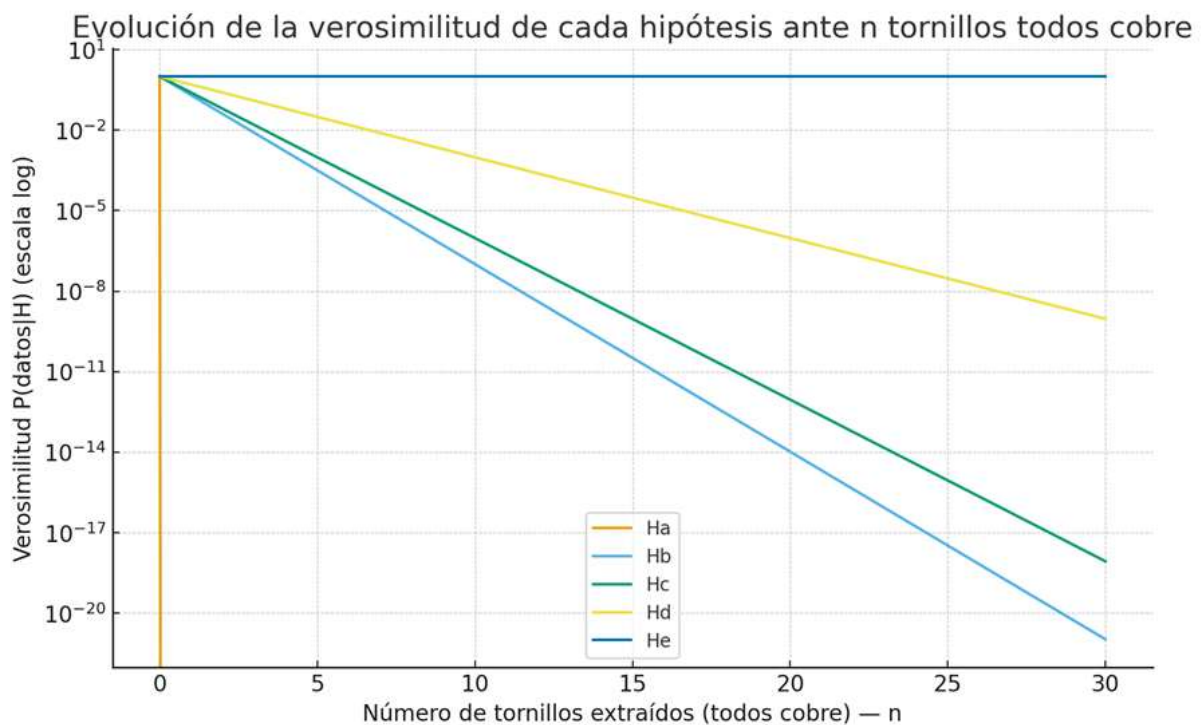
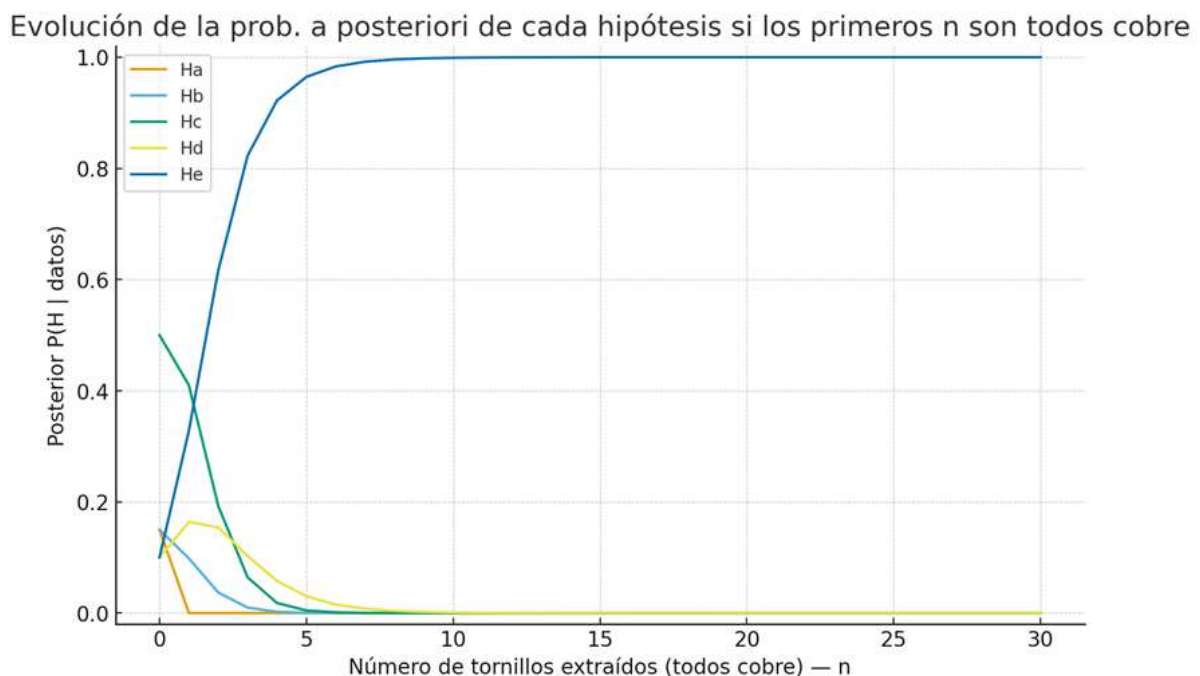


Figura 2: Evolución de las probabilidades a posteriori $P(H|\text{datos})$ si los primeros n son todos cobre.



1.4 Probabilidad de que el cuarto tornillo sea cobre (por hipótesis)

Si no condicionamos en observaciones previas, la probabilidad para una extracción individual es la fracción de cobre de cada hipótesis:

H_a : 0.00

H_b : 0.20

H_c : 0.25

H_d : 0.50

H_e : 1.00

Metodología (breve)

- 1) Se asumió independencia y modelo Bernoulli con parametro p_H por hipótesis.
- 2) Para n observaciones todas cobre, verosimilitud p_H^n .
- 3) Se aplicó Bayes y se normalizó para obtener posteriors.

Conclusión

Después de observar que los 10 primeros tornillos extraídos son todos de cobre, la evidencia favorece extremadamente a H_e (la caja es totalmente de cobre), con posterior ≈ 0.999 .

Ejercicio 2

Aprendizaje supervisado:

- Se entrena el algoritmo con un **conjunto de datos etiquetado** (inputs acompañados de la salida deseada).
- El objetivo es aprender una función que relacione las entradas con las salidas, para luego poder predecir la etiqueta de nuevos datos.
- Ejemplos: clasificación (KNN, árboles de decisión, redes neuronales) y regresión (regresión lineal).

Aprendizaje no supervisado:

- Se entrena el algoritmo con un **conjunto de datos sin etiquetas**.
- El objetivo es descubrir la **estructura oculta** o **patrones** en los datos (agrupar, reducir dimensiones, detectar anomalías).
- Ejemplos: clustering (K-Means), reducción de dimensionalidad (PCA).
- En entornos reales con predominio de incertidumbre los agentes utilizan probabilidades y teoría de la decisión, pero primero, deben aprender sus teorías probabilísticas sobre el mundo a partir de la experiencia.

Ejercicio 3

El programa implementa una **clasificación de puntos en el plano (x,y)** combinando los siguientes algoritmos de aprendizaje :

- **K-means** (para agrupar los puntos sin etiquetas previas).
- **K-Nearest Neighbors (KNN)** (para clasificar nuevos puntos a partir de lo aprendido con K-means).

Todo esto se visualiza en **3 gráficos** y se acompaña con resultados impresos en consola.

1. Generación de los datos

- Se generan **23 puntos** con coordenadas (x, y) en el intervalo [0, 5].
- Los **primeros 20** se usan como **entrenamiento** para K-means.

- Los **3 restantes** se reservan como **puntos de test** para KNN.

En cada ejecución los puntos cambian porque la semilla se deja libre (**None**).

2. Inicialización de centroides

- Se generan **2 centroides iniciales** aleatorios en el mismo intervalo $[0,5]$.
- Estos centroides son el punto de partida del algoritmo K-means.

3. Implementación de K-means

Explicación paso a paso:

1. **Asignación:** cada punto se asigna al cluster cuyo centroide esté más cerca (distancia Euclidiana).
2. **Actualización:** cada centroide se mueve al promedio (media) de los puntos que tiene asignados.
3. **Condición de parada:** si los centroides cambian muy poquito (menos de **tol**), o se llega a **max_iter**, se detiene el algoritmo.

Esto es un **proceso iterativo** que va ajustando los centroides hasta que convergen.

Resultado:

- **labels** → qué cluster le toca a cada uno de los 20 puntos.
- **centroids** → posiciones finales de los centroides.

4. Elección de K para KNN

- El usuario elige entre **tres valores posibles** de vecinos: 1, 3 o 5.
- Según la opción, se guarda en **k**.
- Si elige mal, se asigna por defecto **k = 3**.

5. Clasificación con KNN

1. Se entrena un clasificador **KNN** con los puntos de entrenamiento y las etiquetas que asignó K-means.
2. Luego se usa ese modelo para **clasificar los 3 puntos de test**.
 - Cada punto de test se clasifica según la mayoría de los **K vecinos más cercanos**.

Resultado: **predictions** contiene la clase (0 o 1) de cada punto de test.

6. Visualización con gráficos

Se generan **3 subplots en paralelo**:

(a) Puntos + centroides iniciales

- 20 puntos negros.
- 2 centroides iniciales en rojo.

(b) Clusters finales de K-means

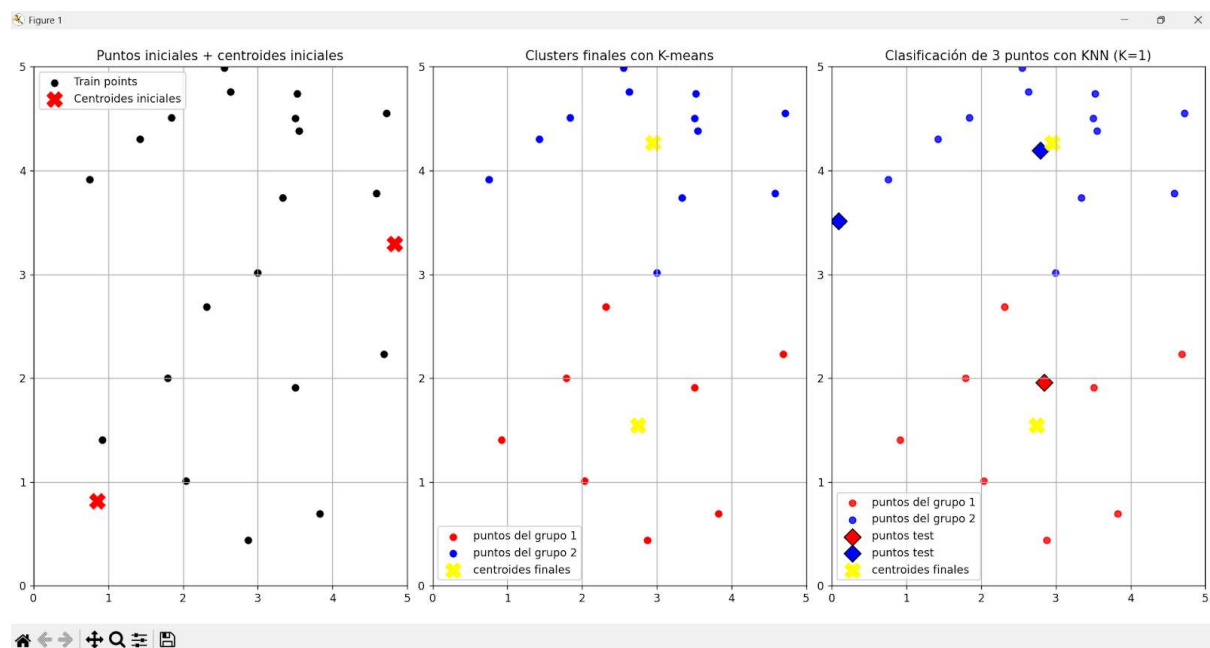
- Los 20 puntos aparecen **coloreados** según su cluster (rojo o azul).
- Los centroides finales aparecen en **amarillo**.

(c) Clasificación con KNN

- Los 20 puntos de entrenamiento siguen coloreados por K-means.
- Los 3 puntos de test aparecen como **diamantes (D)**, coloreados según la clasificación KNN.
- Los centroides finales permanecen en amarillo.

7. Resultados de la ejecución

En la siguiente imagen se pueden observar los pasos hasta llegar al requerimiento final.



- Primer gráfico: se ubican los primeros 20 puntos aleatorios en el plano y se asignan los centroides iniciales.
- Segundo gráfico: en esta imagen se observan los centroides finales (producto de las iteraciones realizadas por el algoritmo K-means) y los puntos asociados a cada grupo, diferenciados en color rojo y azul.
- Tercer gráfico: se añaden los puntos correspondientes al análisis del algoritmo KNN. En la parte superior del gráfico se aclara cuál

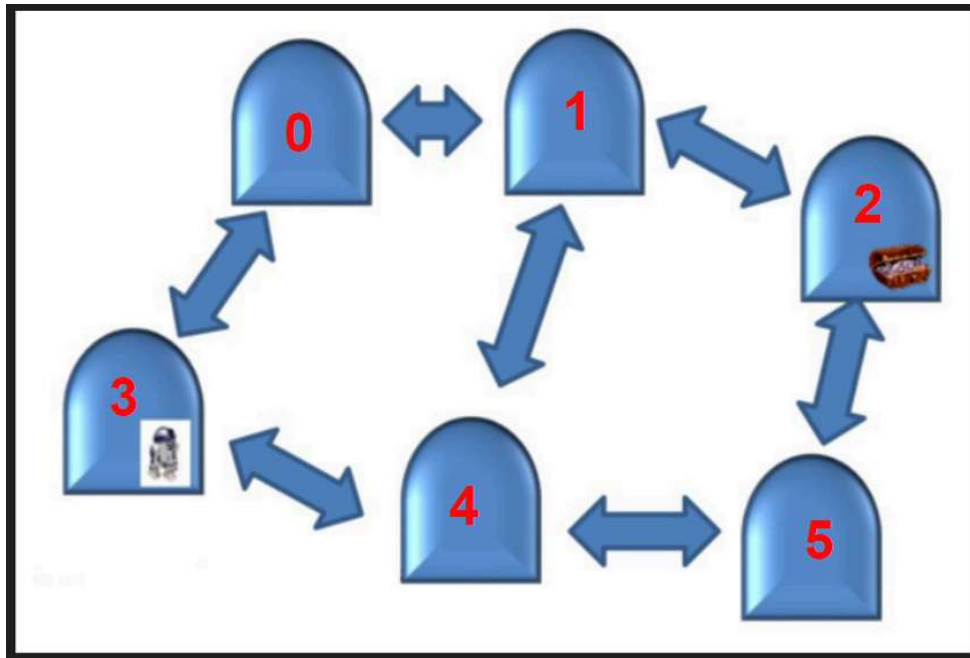
es el valor de K elegido por el usuario (en el caso de la imagen 1). Los rombos representan los 3 puntos evaluados mediante el algoritmo KNN.

Conclusiones

1. **Generación aleatoria:** Cada ejecución genera puntos y centroides distintos, por lo que los resultados cambian cada vez.
2. **K-means:** Es un algoritmo no supervisado que agrupa los datos en 2 clusters, ajustando los centroides hasta que convergen.
3. **KNN:** Clasifica nuevos puntos de test en base a los vecinos más cercanos de los clusters ya formados.
4. **Valor de K:**
 - **K=1** → muy sensible al ruido, puede cambiar por un solo vecino.
 - **K=3** → más robusto, buena opción en este ejemplo.
 - **K=5** → suaviza aún más, pero con pocos datos puede “mezclar” clusters.
5. **Visualización:** Los 3 gráficos permiten entender todo el proceso:
 - Cómo arranca con centroides iniciales.
 - Cómo quedan los clusters al finalizar K-means.
 - Cómo se clasifican los puntos de test con KNN.

Ejercicio 4:

Usamos la siguiente numeración para las salas:

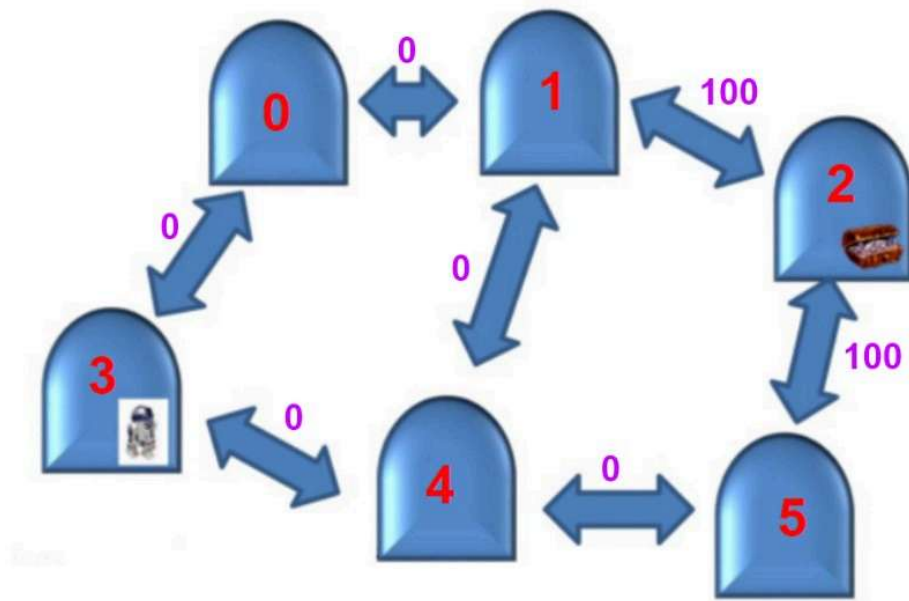


1. Contexto del Problema

El ejercicio plantea un entorno representado por **salas interconectadas** (estados). El objetivo es llegar a la **sala del tesoro** (estado 2). Se nos pide implementar un algoritmo de **Q-Learning** con factor de descuento ($\gamma = 0.9$), donde:

- Cada transición válida recibe recompensa **0**.
- Las transiciones que llevan directamente a la sala del tesoro reciben recompensa **100**.
- Movimientos no permitidos se penalizan con **-1** (no disponibles).

El inciso 4.1 pide graficar estos valores sobre la imagen de la consigna:



La ecuación utilizada en el aprendizaje Q es la **ecuación de Bellman para Q-Learning**:

$$Q(s,a) = R(s,a) + \gamma \max_{A} Q(s',A)$$

Donde:

- (s): estado actual
- (a): acción (moverse a otra sala)
- (R(s,a)): recompensa inmediata de realizar (a) en (s)
- (s'): nuevo estado alcanzado
- (A): conjunto de acciones desde (s')
- (\gamma): factor de descuento que valora las recompensas futuras.

2. Herramientas Utilizadas

El ejercicio fue resuelto en **Python** utilizando las siguientes librerías:

- **NumPy**: para manejar matrices (R y Q) y realizar operaciones matemáticas.
- **Matplotlib**: para graficar resultados y visualizar la política óptima.
- **NetworkX**: para representar el grafo de salas y sus conexiones.

3. Etapas de la Resolución

a) Definición del entorno

- Se identificaron las **6 salas** (estados 0 a 5).
- Se establecieron las **adyacencias** (qué salas conectan con cuáles) según el diagrama provisto.
- Se definió la sala **2 como objetivo (tesoro)**.

b) Construcción de la matriz de recompensas R

- Dimensión: 6x6.
- Inicialmente se llenó con **-1** para marcar transiciones no válidas.
- Se asignó:
 - **0** para transiciones válidas que no llevan al tesoro.
 - **100** para transiciones que llevan al tesoro (ej: de 1 a 2, o de 5 a 2).

c) Inicialización de la matriz Q

- Una matriz de 6x6 con todos los valores en **0** al inicio.

- Representa el conocimiento del agente sobre la utilidad de cada acción.

d) Iteraciones de aprendizaje (Q-Learning)

- Se aplica la fórmula de Bellman para cada transición válida.
- Se actualizan los valores de Q hasta la **convergencia** (cuando los cambios son menores a una tolerancia definida). Esto se mide mediante una variable “delta” que representa cuánto cambia la matriz Q en una iteración completa. Específicamente es el cambio máximo. Esto se aclara ya que “delta” será un valor importante cuando se plantea otro método (al final del informe).

Para hacer las iteraciones se utilizó un bucle while true con dos for anidados que van recorriendo las matriz R en cada iteración y recalculando los valores de Q hasta la convergencia. La lógica de las iteraciones se muestra a continuación.

```
Inicio:
- Q = todos ceros

Iteración 1:
- Para cada s, a válidos:
   $Q[s,a] = R[s,a] + \gamma * 0$ 
  =>  $Q[s,a] = R[s,a]$ 
  => Solo recompensas inmediatas

Iteración 2:
- Ahora Q tiene valores  $\neq 0$ 
- Nuevos  $Q[s,a] = R[s,a] + \gamma * \max(Q[s', -])$ 
  => Incorporamos valor futuro
  => Mejores decisiones

Iteraciones siguientes:
- Se propagan valores óptimos por el grafo
- Cada vez delta (cambio) es menor
- Cuando  $\text{delta} < \text{tol}$ , paramos
```

e) Obtención de la matriz Q óptima

- Una vez convergido, se obtiene la matriz con los valores $Q(s,a)$.
- Estos valores indican la utilidad esperada de tomar cada acción.

f) Normalización

- Se divide cada valor de Q por el valor máximo encontrado, quedando todos los valores en el rango $[0,1]$.

g) Política óptima

- Para cada estado, se elige la acción con mayor $Q(s,a)$.

- Esto define la política que guía al agente desde cualquier sala hacia el tesoro.

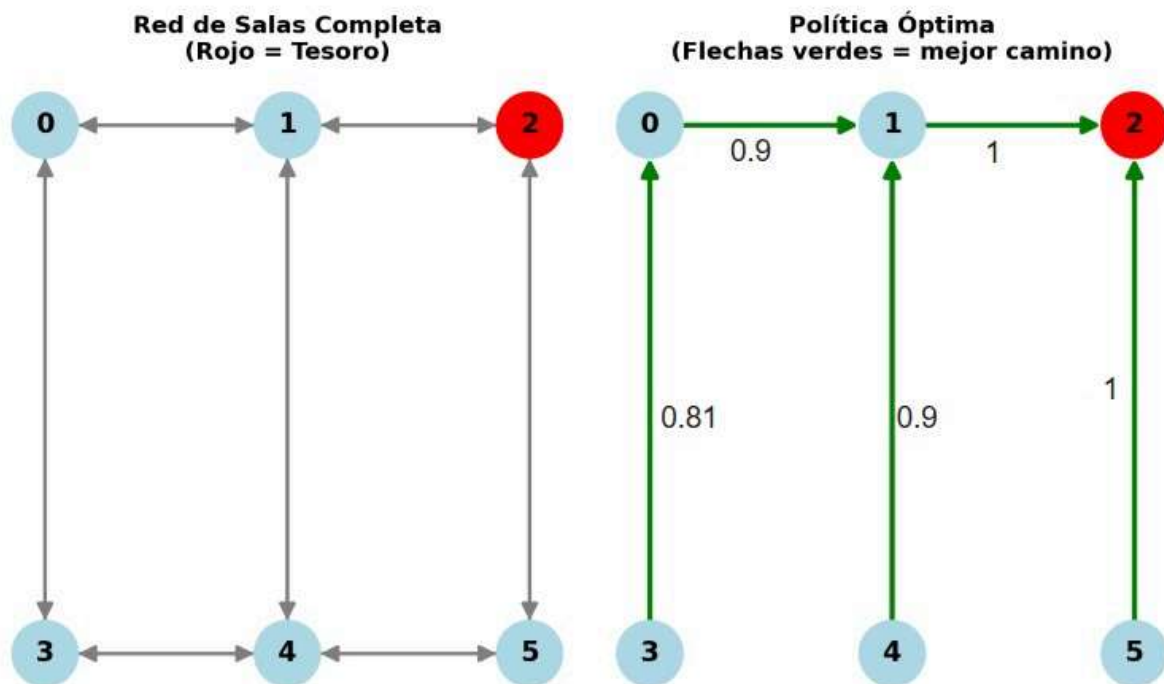
h) Visualización

- Se muestran dos grafos:
 - **Red completa** de salas con todas las conexiones.
 - **Política óptima** marcada con flechas verdes hacia el tesoro.

4. Resultados

- La matriz R refleja correctamente las reglas del problema.
- La matriz Q converge a valores estables que indican los caminos óptimos.
- La política extraída guía al agente hacia el tesoro desde cualquier sala, siguiendo el menor número de pasos.

El programa grafica la situación original y la política obtenida luego de aplicar Q-learning. Este gráfico es el siguiente (se agregaron los valores normalizados de Q).



Además el programa muestra por terminal todo lo siguiente:

- Matrices R, Q (original con todos ceros), Q final y Q normalizada .
- Las variaciones máximas de Q de una iteración a la siguiente cada 10 iteraciones. Se observa cómo va convergiendo a cero.
- La cantidad máxima de iteraciones y su variación correspondiente.
- La política óptima y cuál sería el camino a seguir dependiendo de la sala que inicie.

5. Conclusión

Este ejercicio permitió aplicar **Q-Learning** en un entorno simple de grafos. Se utilizaron herramientas de programación científica para construir las matrices de recompensas y aprendizaje, y para visualizar los resultados. El agente, a través de iteraciones, aprendió la mejor estrategia (política óptima) para llegar al tesoro desde cualquier sala, maximizando la recompensa acumulada.

Otro criterio:

Existe otra fórmula para Q-learning que es un poco más conocida. De hecho, esta otra fórmula se puede encontrar en el libro “Inteligencia Artificial: un enfoque moderno de Russell y Norvig, segunda edición, página 881 (fórmula 21.8)”.

A continuación se hace una comparación entre la fórmula original utilizada (“método 1”) y la nueva fórmula que se propone (“método 2”).

1. Fórmulas

Método 1 (Fórmula 1):

$$Q(s, a) = R(s, a) + \gamma \cdot \max_A Q(s', A)$$

Método 2 (Fórmula 2):

$$Q(s, a) = Q(s, a) + \alpha \cdot \left(R(s, a) + \gamma \cdot \max_A Q(s', A) - Q(s, a) \right)$$

2. Parámetros y su interpretación

- **Q(s,a):** valor estimado de realizar la acción **a** en el estado **s**.
- **R(s,a):** recompensa inmediata obtenida al realizar la acción **a** en el estado **s**.
- **γ (gamma):** factor de descuento. Indica cuánto valoramos las recompensas futuras respecto a las inmediatas.
 - $0 \leq \gamma < 1$
 - γ cercano a 1: el agente planea a largo plazo.

- γ cercano a 0: el agente se enfoca en recompensas inmediatas.
 - **α (alfa):** tasa de aprendizaje. Define cuánto se ajusta el valor $Q(s,a)$ con la nueva información.
 - $0 < \alpha \leq 1$
 - α grande: el agente da mucho peso a la experiencia reciente.
 - α pequeño: el agente aprende más lento, ponderando más la experiencia acumulada.
-

3. Relación entre ambas fórmulas

- **Fórmula 1:** Es una versión más simple, similar a la **ecuación de Bellman determinista**. Actualiza directamente el valor $Q(s,a)$ como si tuviéramos toda la información del entorno y quisiéramos resolver el sistema de ecuaciones de manera exacta.
 - **Fórmula 2:** Es la **regla de actualización clásica de Q-learning**. Introduce el parámetro de tasa de aprendizaje (α), lo que permite hacer actualizaciones **incrementales** a partir de experiencias individuales.
 - **Relación:** La fórmula 1 puede verse como un **caso particular de la fórmula 2** con ($\alpha = 1$). En ese caso, el valor antiguo de $Q(s,a)$ no influye y se reemplaza por el nuevo cálculo.
-

4. Cuándo usar cada una

- **Fórmula 1 (sin alfa):**

- Adecuada en **ejemplos pequeños y deterministas** donde podemos recorrer todas las transiciones y actualizar la matriz Q de forma síncrona.
- Es más cercana a un método de **programación dinámica** que a aprendizaje por refuerzo puro.

- **Fórmula 2 (con alfa):**

- Usada en la práctica en **entornos grandes, estocásticos o desconocidos**, donde el agente aprende por interacción.
- Permite ajustar progresivamente los valores de Q sin necesidad de tener toda la información del entorno desde el principio.

5. Interpretación de α y γ

- **α (tasa de aprendizaje):**

- **Grande (cercano a 1):** el agente aprende rápido de experiencias recientes, pero puede volverse inestable si el entorno es ruidoso.
- **Pequeño (cercano a 0):** el agente es más estable y necesita más tiempo para aprender, ya que acumula la experiencia lentamente.

En nuestro ejercicio se utilizó un alfa de 0.1, lo cual es un valor típico, pero implica que la convergencia será más lenta.

💡 ¿Por qué usar α pequeño entonces?

En problemas **reales** donde:

- No conocemos R completamente de antemano
- El entorno es estocástico (con ruido/incertidumbre)
- Necesitamos explorar

El α pequeño ayuda a:

- **Estabilidad:** No sobre-reaccionar a experiencias únicas
- **Promediado:** Suavizar el ruido en las recompensas
- **Adaptabilidad:** Seguir aprendiendo gradualmente

En **nuestro ejercicio** (R conocida, determinista), el método directo es más eficiente porque podemos calcular todo de una vez sin necesidad de aprendizaje gradual.

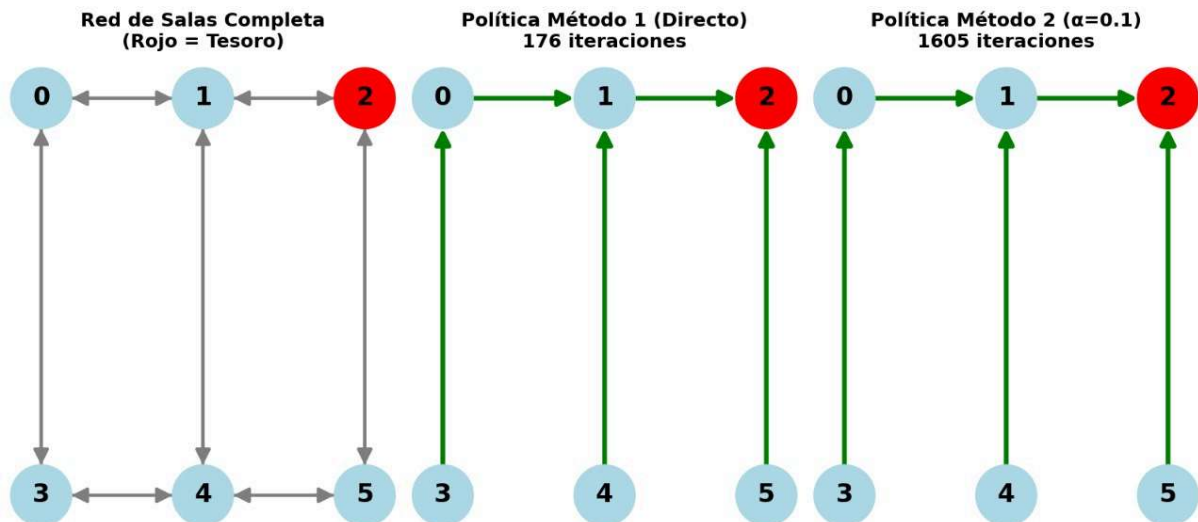
- γ (factor de descuento):
 - **Grande (cercano a 1):** se planifica a largo plazo, valorando recompensas futuras.
 - **Pequeño (cercano a 0):** el agente se enfoca en recompensas inmediatas, ignorando consecuencias futuras.

6. Conclusión

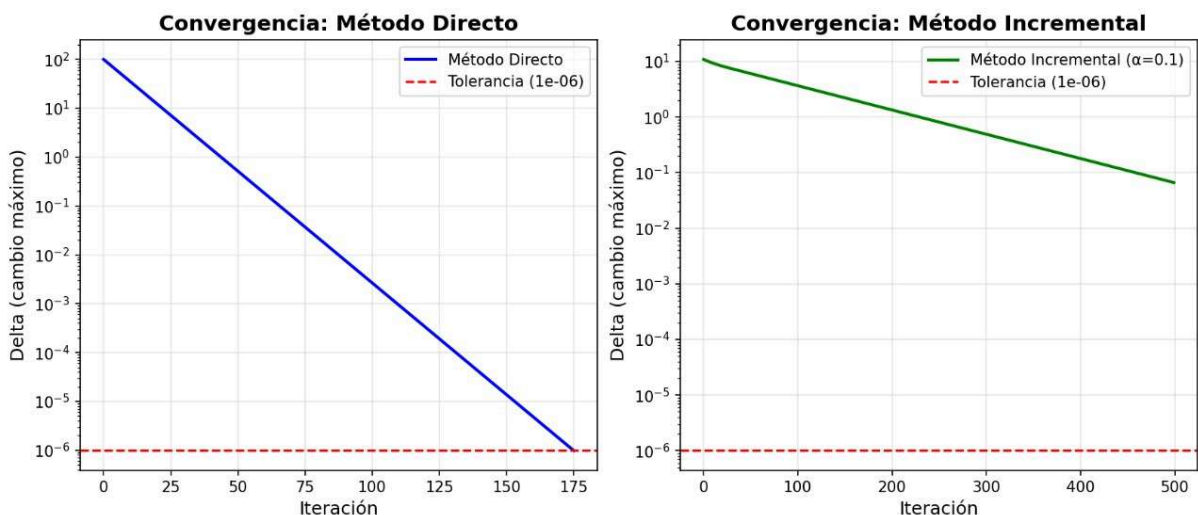
- La **fórmula 1** es útil como método de referencia o en problemas simples y completamente conocidos.
- La **fórmula 2** es el estándar en Q-learning, ya que incorpora (α) para controlar la velocidad y estabilidad del aprendizaje.
- En resumen, la **fórmula 1** es un caso particular de la fórmula 2 con ($\alpha = 1$).

Inteligencia Artificial 2025 - Grupo 5

Si bien ambos métodos llegan al mismo resultado, se ve una clara diferencia de velocidad de convergencia. El método 2 se demora casi diez veces más en converger (debido al pequeño alfa elegido).



Se muestra una gráfica comparativa que muestra cómo se reduce delta al avanzar las iteraciones. Claramente, el método 1 (método directo en azul) es más brusco y directo que el método 2 (método incremental en verde).



Sobre todo en la primera iteración es donde más claro es el efecto de incorporar alfa. Para la iteración 1, el delta correspondiente para el método 1 es 100. Esto significa que en esta iteración, el máximo cambio de Q fue de 100 (unidades). Pero para el método 2, delta vale 10, lo

Inteligencia Artificial 2025 - Grupo 5

cual es un cambio máximo en Q (en esa iteración) de 10 solamente. El valor delta se puede interpretar de cierta manera como un velocímetro del aprendizaje. Esto hace evidente que el método 2 demorará mucho más en llegar al mismo valor de Q obtenido por el método 1.

Aspecto	Método 1 (Directo)	Método 2 ($\alpha=0.1$)
Delta Inicial	Grande (100)	Pequeño (10)
Razón	Actualización completa	Solo aprende 10% por paso
Iteraciones	Pocas (~10-20)	Muchas (100-1000+)
Convergencia	Rápida	Gradual
Mejor para	Problemas deterministas conocidos	Problemas con exploración/ruido

Todo lo aquí expuesto parece sugerir que el método 1 es superior al 2, pero hay que recordar que el método 1 tiene serias limitaciones en los problemas reales. En estos no solemos conocer perfectamente R y el entorno suele tener incertidumbres y ruidos. En estos casos se prefiere un método más lento pero más estable, que pueda adaptarse gradualmente al entorno.